

رسالة محمد



یادگیری ماشین

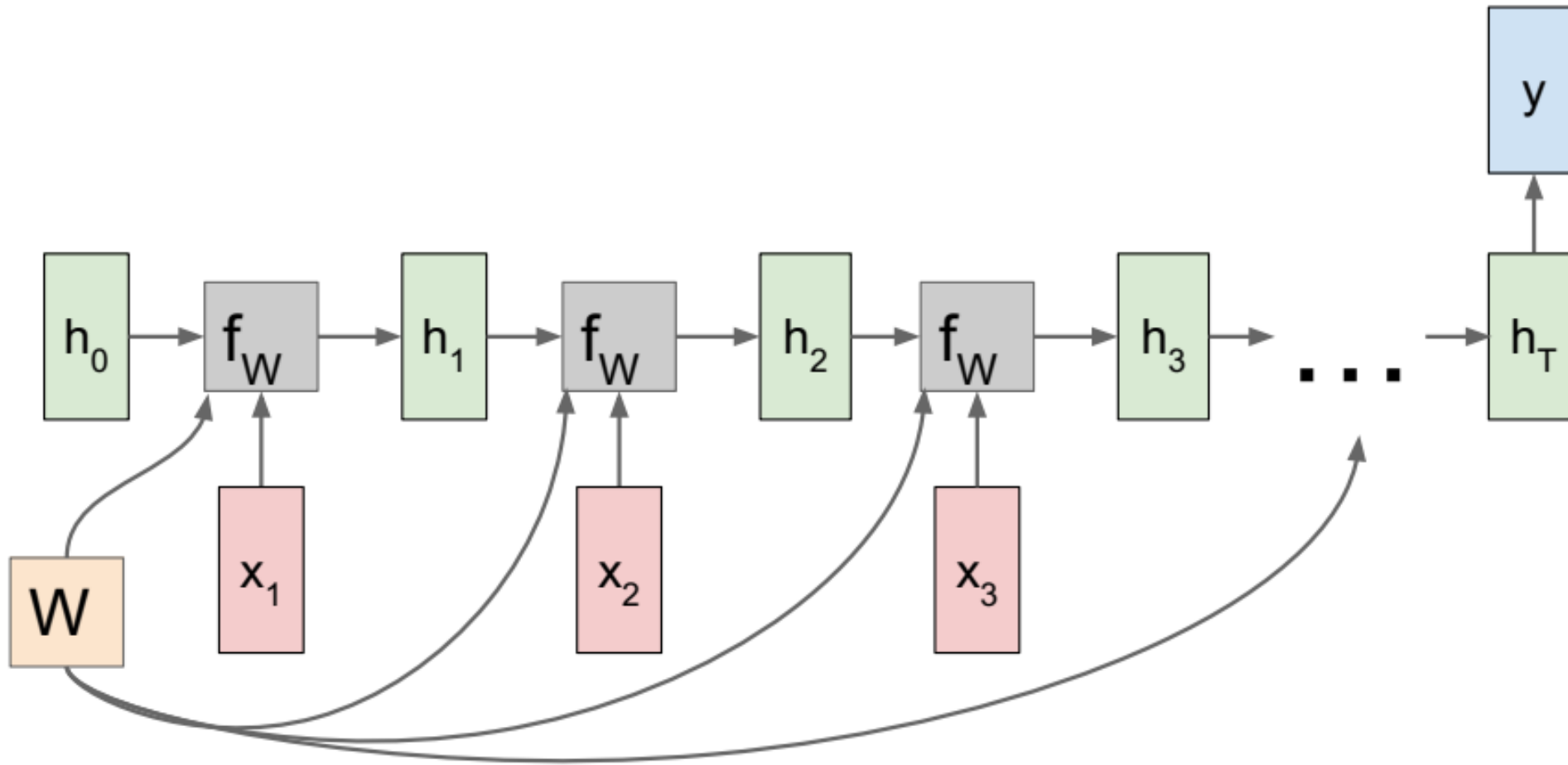
مدرس: محمدرضا محمدی

بهار ۱۴۰۲

شبکه‌های عصبی بازگشتی

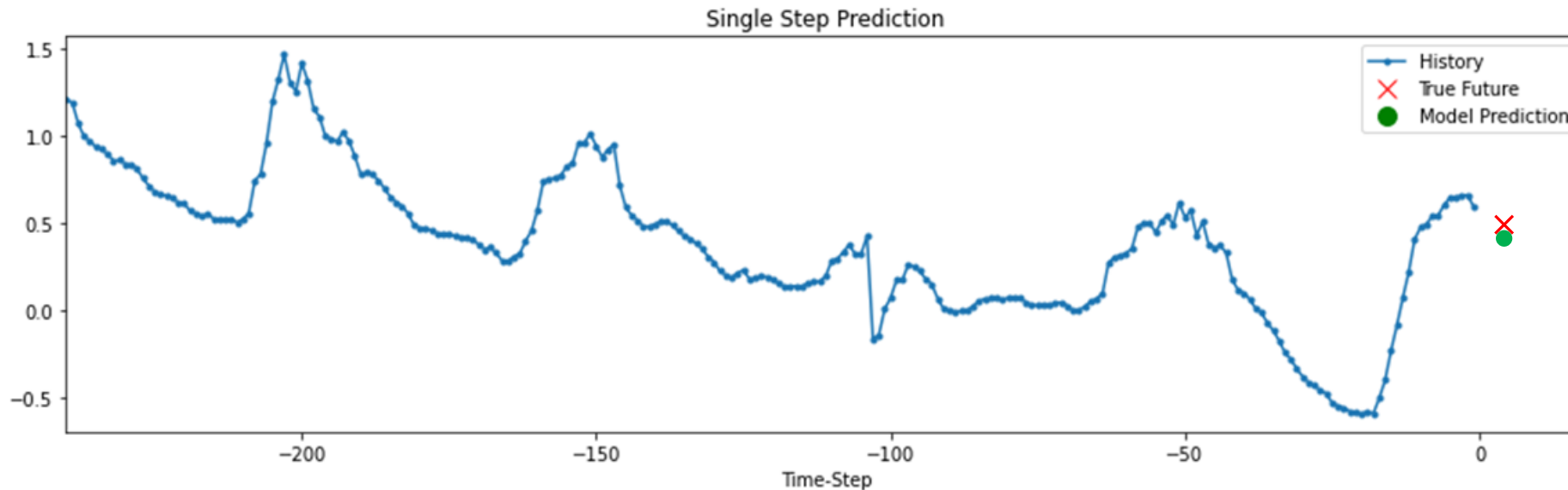
Recurrent Neural Networks

Many to One



مثال: پیش‌بینی سری زمانی

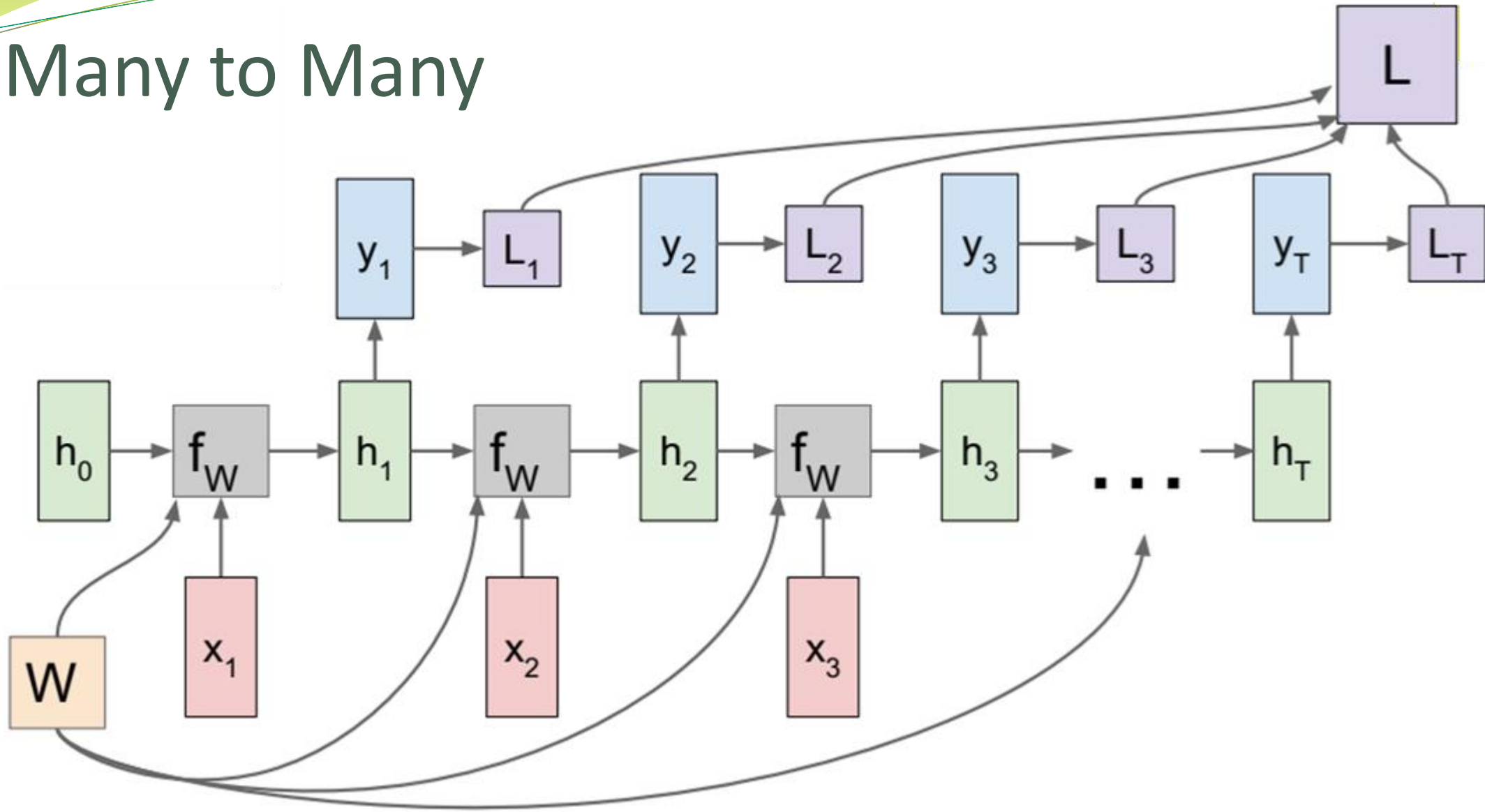
- از داده‌های ۷۲۰ زمان گذشته ($720/6 = 120$ ساعت) با گام ۳ برای پیش‌بینی استفاده می‌شود
- این داده‌ها برای پیش‌بینی دما پس از N گام زمانی ($N/6$ ساعت) استفاده می‌شود



شبکه	تعداد پارامتر	زمان هر epoch	خطای آموزش	خطای آزمون
FC (64) + FC(1)	107,649	52s	0.1377	0.1413
SimpleRNN (64) + FC (1)	4,673	241s	0.0403	0.0599

زمان هدف		۱ ساعت		۶ ساعت		۱۲ ساعت	
شبکه		خطای آموزش	خطای آزمون	خطای آموزش	خطای آزمون	خطای آموزش	خطای آزمون
SimpleRNN (64) + FC (1)		0.0158	0.0311	0.2193	0.2516	0.2752	0.4135
GRU (64) + FC (1)		0.0127	0.0165	0.0662	0.0831	0.0986	0.1313

Many to Many



مدل زبان

- هدف از یک مدل زبان این است که احتمال یک دنباله از توکن‌ها را تخمین بزند

$$P(x_1, x_2, \dots, x_T)$$

- یک از کاربردهای مدل‌های زبان، تولید متن طبیعی است

$$x_t \sim P(x_t \mid x_{t-1}, \dots, x_1)$$

- علاوه بر این، برای ایجاد یک گفتگوی معنادار، می‌توان از شرطی کردن متن به قطعات گفتگوی قبلی استفاده کرد

- به عنوان مثال، عبارت‌های “to recognize speech” و “to wreck a nice beach” شبیه به هم تلفظ می‌شوند اما از لحاظ محتوایی و با استفاده از مدل زبان می‌توان عبارت نخست را انتخاب کرد

یادگیری یک مدل زبان

- طبق قاعده ضرب در تئوری احتمال:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{t-1}, \dots, x_1)$$

- به عنوان مثال:

$$P(\text{deep, learning, is, fun}) = P(\text{deep})P(\text{learning} | \text{deep})P(\text{is} | \text{deep, learning})P(\text{fun} | \text{deep, learning, is})$$

- نیاز است تا احتمال واژه‌ها و همچنین احتمال شرطی واژه‌ها را محاسبه کنیم
- اگر یک مجموعه داده بسیار بزرگ داشته باشیم، می‌توانیم به صورت آماری آنها را تخمین بزنیم
- برای کلمه “deep” می‌توان درصد جملاتی که با آن شروع شده‌اند را محاسبه کرد
- یک روش با مقداری دقت کمتر این است که مجموع تمام رخدادها را به تعداد کل واژگان تقسیم کنیم

یادگیری یک مدل زبان

- برای تخمین احتمال شرطی می توان به صورت زیر عمل کرد

$$\hat{P}(\text{learning} \mid \text{deep}) = \frac{n(\text{deep, learning})}{n(\text{deep})}$$

- تخمین احتمال شرطی دشوارتر است زیرا تعداد تکرار جفت واژه ها کمتر است و این پیچیدگی برای دنباله های طولانی تر بیشتر می شود

$$\hat{P}(x) = \frac{n(x) + \epsilon_1/m}{n + \epsilon_1}$$

$$\hat{P}(x' \mid x) = \frac{n(x, x') + \epsilon_2 \hat{P}(x')}{n(x) + \epsilon_2}$$

$$\hat{P}(x'' \mid x, x') = \frac{n(x, x', x'') + \epsilon_3 \hat{P}(x'')}{n(x, x') + \epsilon_3}$$

- به دلیل اینکه برخی ترکیب ها ممکن است در مجموعه داده ما اتفاق نیافتاده باشند، در بسیاری مواقع از تقریب هایی مانند Laplace smoothing استفاده می کنیم - اگر $\epsilon_1 = 0$ باشد همان حالت عادی است و اگر $\epsilon_1 \rightarrow \infty$ به توزیع یکنواخت همگرا می شود

مدل‌های مارکوف

- فرض می‌کنیم نمونه فعلی به تعداد کمی از نمونه‌های قبلی وابسته باشد
 - مرتبه ۰ (مستقل):

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2)P(x_3)P(x_4)$$

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2|x_1)P(x_3|x_2)P(x_4|x_3)$$

- مرتبه ۱:

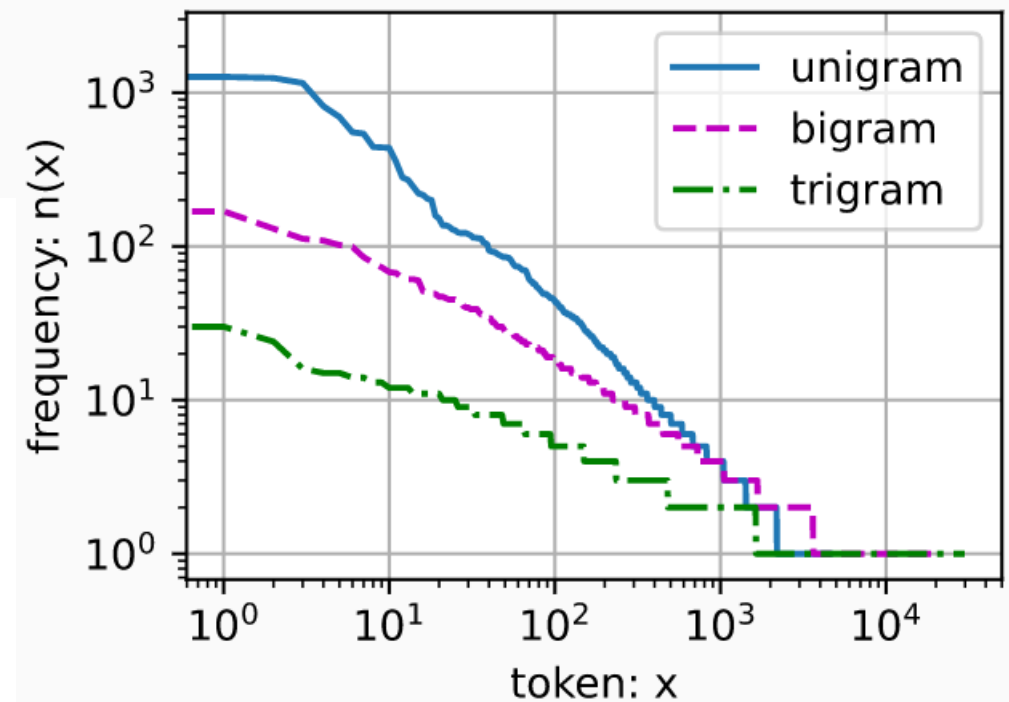
$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)P(x_4|x_2, x_3)$$

- مرتبه ۲:

- به این مدل‌ها به ترتیب unigram، bigram و trigram گفته می‌شود

مثال: مجموعه داده The Time Machine

```
[('the', 2261),  
 ('i', 1267),  
 ('and', 1245),  
 ('of', 1155),  
 ('a', 816),  
 ('to', 695),  
 ('was', 552),  
 ('in', 541),  
 ('that', 443),  
 ('my', 440)]  
[[('of', 'the'), 309],  
 (('in', 'the'), 169),  
 (('i', 'had'), 130),  
 (('i', 'was'), 112),  
 (('and', 'the'), 109),  
 (('the', 'time'), 102),  
 (('it', 'was'), 99),  
 (('to', 'the'), 85),  
 (('as', 'i'), 78),  
 (('of', 'a'), 73)]  
[[('the', 'time', 'traveller'), 59],  
 (('the', 'time', 'machine'), 30),  
 (('the', 'medical', 'man'), 24),  
 (('it', 'seemed', 'to'), 16),  
 (('it', 'was', 'a'), 15),  
 (('here', 'and', 'there'), 15),  
 (('seemed', 'to', 'me'), 14),  
 (('i', 'did', 'not'), 14),  
 (('i', 'saw', 'the'), 13),  
 (('i', 'began', 'to'), 13)]
```



یادگیری یک مدل زبان

- برای تخمین احتمال شرطی می توان از حالت پنهان استفاده کرد

$$P(x_t \mid x_{t-1}, \dots, x_1) \approx P(x_t \mid h_{t-1})$$

$$h_t = f(x_t, h_{t-1})$$

- با استفاده از یادگیری عمیق می توانیم مدلی آموزش دهیم که بتواند احتمال توکن بعدی را پیش بینی کند

مثال: مدل زبانی سطح کاراکتر

- دنباله آموزشی نمونه:

- "hello"

input chars: "h" "e" "l" "l"

مثال: مدل زبانی سطح کاراکتر

target chars: "e" "l" "l" "o"

• دنباله آموزشی نمونه:

- "hello"

input chars: "h" "e" "l" "l"

مثال: مدل زبانی سطح کاراکتر

target chars: "e" "l" "l" "o"

- دنباله آموزشی نمونه:

- "hello"

- توکن‌ها:

- [h,e,l,o]

input layer

1
0
0
0

0
1
0
0

0
0
1
0

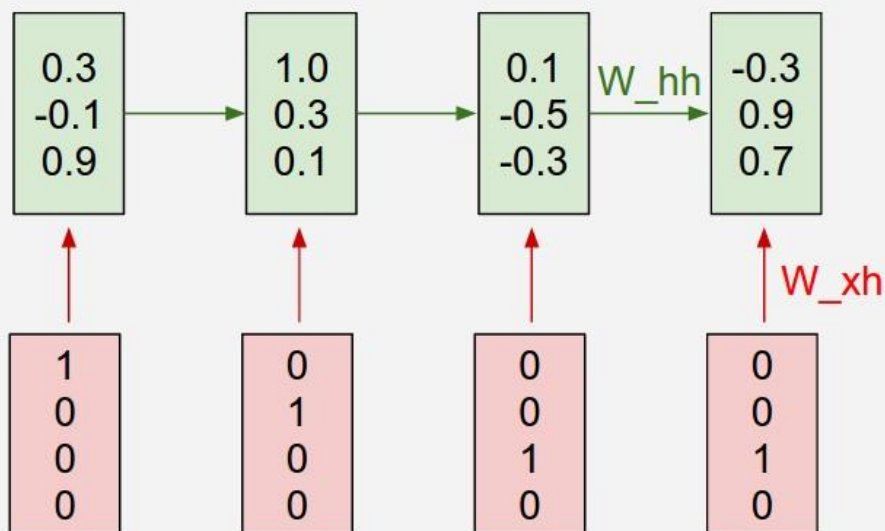
0
0
1
0

input chars: "h" "e" "l" "l"

مثال: مدل زبانی سطح کاراکتر

target chars: "e" "l" "l" "o"

hidden layer



input layer

input chars: "h" "e" "l" "l"

- دنباله آموزشی نمونه:

"hello" -

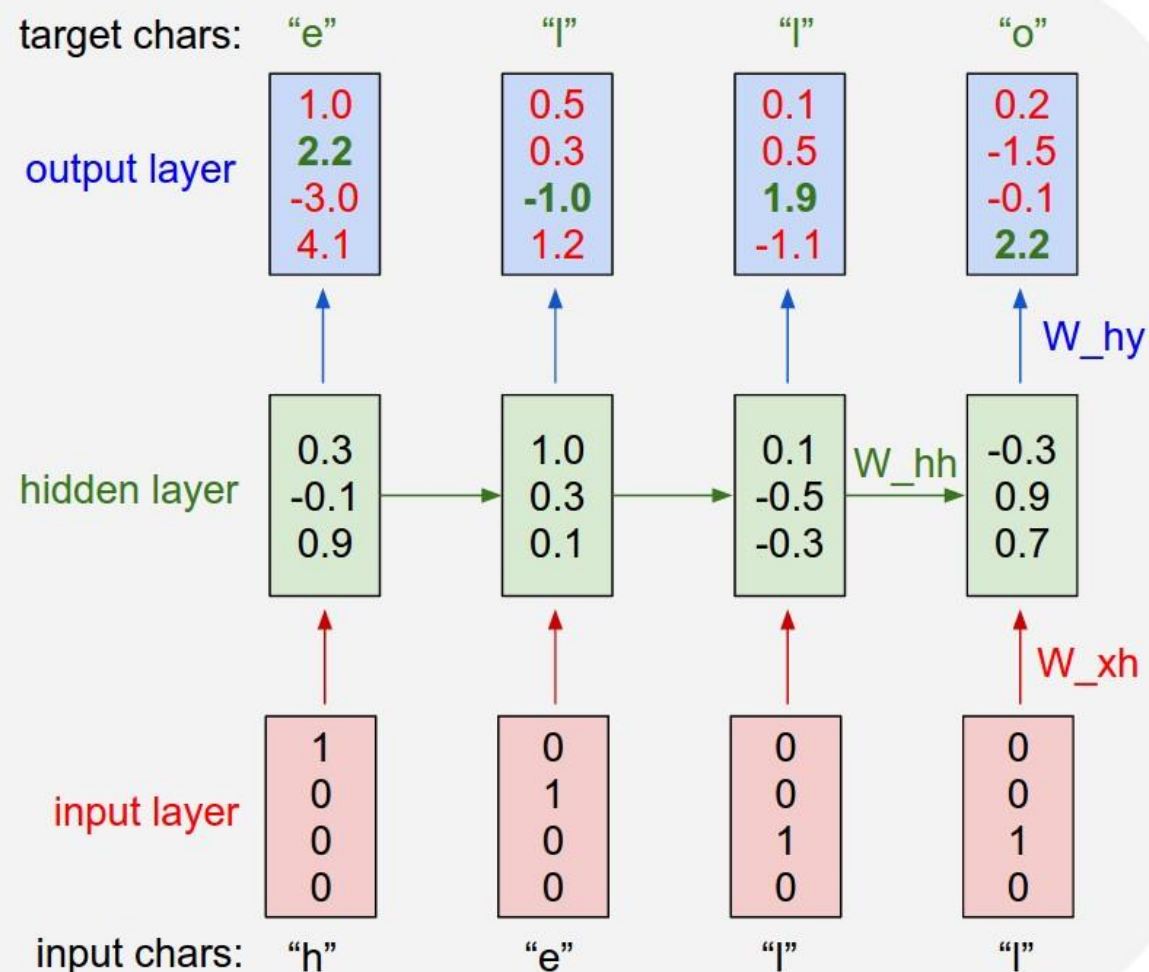
- توکن‌ها:

[h,e,l,o] -

- لایه بازگشتی میانی:

$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) -$$

مثال: مدل زبانی سطح کاراکتر



- دنباله آموزشی نمونه:

“hello” -

- توکن‌ها:

[h,e,l,o] -

- لایه بازگشتی میانی:

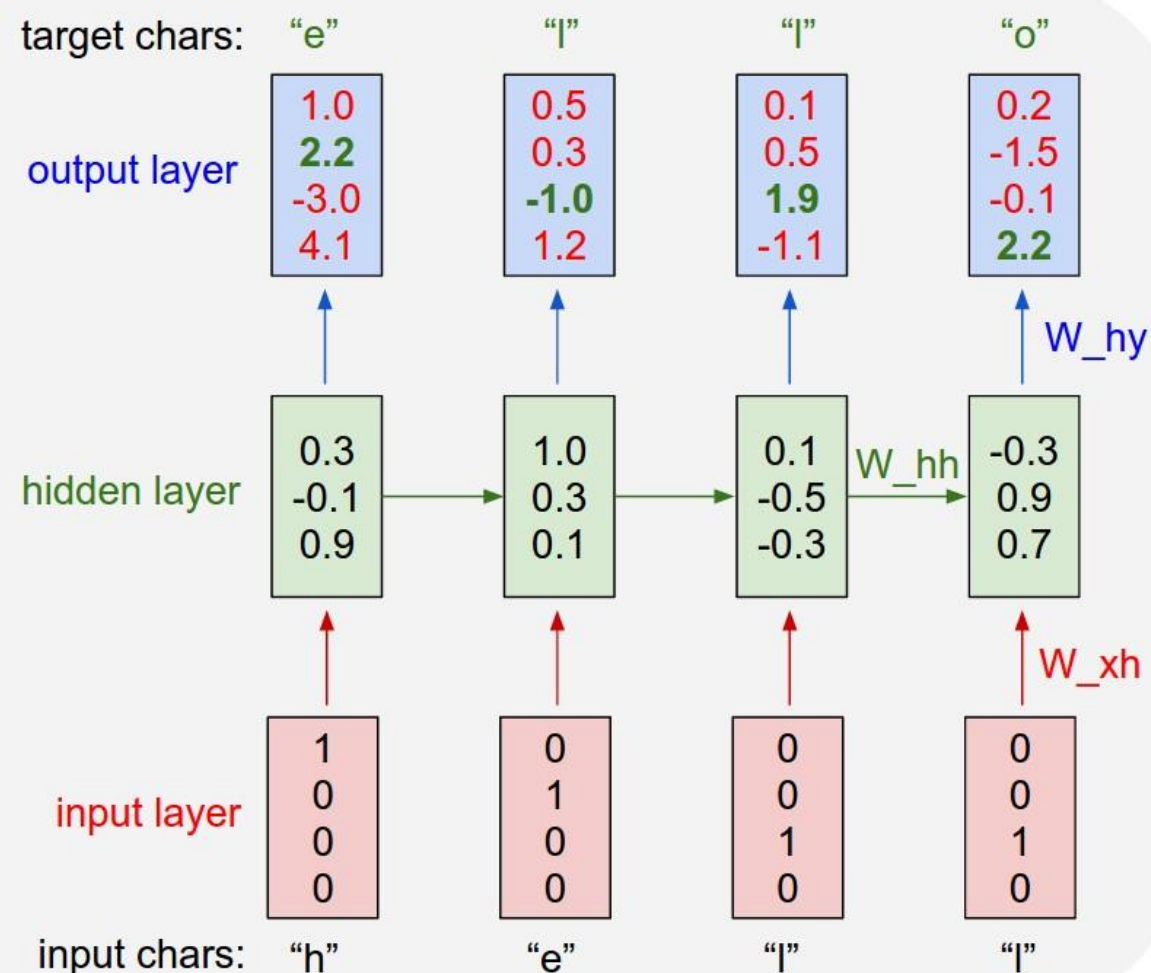
$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) -$$

- لایه کاملاً متصل خروجی:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t -$$

- می‌توان از SoftMax هم استفاده کرد

مثال: مدل زبانی سطح کاراکتر



• در زمان تست:

- در هر گام یک کاراکتر نمونه برداری می شود و ورودی گام بعد می شود

min-char-rnn.py

```

1  """
2  Minimal character-level Vanilla RNN model. Written by Andrej Karpathy (@karpathy)
3  BSD License
4  """
5  import numpy as np
6
7  # data I/O
8  data = open('input.txt', 'r').read() # should be simple plain text file
9  chars = list(set(data))
10 data_size, vocab_size = len(data), len(chars)
11 print 'data has %d characters, %d unique.' % (data_size, vocab_size)
12 char_to_ix = { ch:i for i,ch in enumerate(chars) }
13 ix_to_char = { i:ch for i,ch in enumerate(chars) }
14
15 # hyperparameters
16 hidden_size = 100 # size of hidden layer of neurons
17 seq_length = 25 # number of steps to unroll the RNN for
18 learning_rate = 1e-1
19
20 # model parameters
21 wxh = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
22 whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
23 why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
24 bh = np.zeros((hidden_size, 1)) # hidden bias
25 by = np.zeros((vocab_size, 1)) # output bias
26
27 def lossFun(inputs, targets, hprev):
28     """
29     inputs, targets are both list of integers.
30     hprev is Hx1 array of initial hidden state
31     returns the loss, gradients on model parameters, and last hidden state
32     """
33     xs, hs, ys, ps = {}, {}, {}, {}
34     hs[-1] = np.copy(hprev)
35     loss = 0
36     # forward pass
37     for t in xrange(len(inputs)):
38         xs[t] = np.zeros((vocab_size,1)) # encode in 1-of-k representation
39         xs[t][inputs[t]] = 1
40         hs[t] = np.tanh(np.dot(wxh, xs[t]) + np.dot(whh, hs[t-1]) + bh) # hidden state
41         ys[t] = np.dot(why, hs[t]) + by # unnormalized log probabilities for next chars
42         ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next chars
43         loss += -np.log(ps[t][targets[t],0]) # softmax (cross-entropy loss)
44     # backward pass: compute gradients going backwards
45     dwxh, dwhh, dwhy = np.zeros_like(wxh), np.zeros_like(whh), np.zeros_like(why)
46     dbh, dby = np.zeros_like(bh), np.zeros_like(by)
47     dhnext = np.zeros_like(hs[0])
48     for t in reversed(xrange(len(inputs))):
49         dy = np.copy(ps[t])
50         dy[targets[t]] -= 1 # backprop into y. see http://cs231n.github.io/neural-networks-case-study/#grad if confused here
51         dwhy += np.dot(dy, hs[t].T)
52         dby += dy
53         dh = np.dot(why.T, dy) + dhnext # backprop into h
54         dhraw = (1 - hs[t] * hs[t]) * dh # backprop through tanh nonlinearity

```

```

55     dbh += dhraw
56     dwxh += np.dot(dhraw, xs[t].T)
57     dwhh += np.dot(dhraw, hs[t-1].T)
58     dhnext = np.dot(whh.T, dhraw)
59     for dparam in [dwxh, dwhh, dwhy, dbh, dby]:
60         np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
61     return loss, dwxh, dwhh, dwhy, dbh, dby, hs[len(inputs)-1]
62
63 def sample(h, seed_ix, n):
64     """
65     sample a sequence of integers from the model
66     h is memory state, seed_ix is seed letter for first time step
67     """
68     x = np.zeros((vocab_size, 1))
69     x[seed_ix] = 1
70     ixes = []
71     for t in xrange(n):
72         h = np.tanh(np.dot(wxh, x) + np.dot(whh, h) + bh)
73         y = np.dot(why, h) + by
74         p = np.exp(y) / np.sum(np.exp(y))
75         ix = np.random.choice(range(vocab_size), p=p.ravel())
76         x = np.zeros((vocab_size, 1))
77         x[ix] = 1
78         ixes.append(ix)
79     return ixes
80
81 n, p = 0, 0
82 mwxx, mwxxh, mwxy = np.zeros_like(wxh), np.zeros_like(whh), np.zeros_like(why)
83 mbh, mby = np.zeros_like(bh), np.zeros_like(by) # memory variables for Adagrad
84 smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
85 while True:
86     # prepare inputs (we're sweeping from left to right in steps seq_length long)
87     if p+seq_length+1 >= len(data) or n == 0:
88         hprev = np.zeros((hidden_size,1)) # reset RNN memory
89         p = 0 # go from start of data
90     inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
91     targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
92
93     # sample from the model now and then
94     if n % 100 == 0:
95         sample_ix = sample(hprev, inputs[0], 200)
96         txt = ''.join(ix_to_char[ix] for ix in sample_ix)
97         print '----\n %s \n----' % (txt, )
98
99     # forward seq_length characters through the net and fetch gradient
100     loss, dwxh, dwhh, dwhy, dbh, dby, hprev = lossFun(inputs, targets, hprev)
101     smooth_loss = smooth_loss * 0.999 + loss * 0.001
102     if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
103
104     # perform parameter update with Adagrad
105     for param, dparam, mem in zip([dwxh, dwhh, dwhy, dbh, dby],
106                                   [dwxh, dwhh, dwhy, dbh, dby],
107                                   [mwxx, mwxxh, mwxy, mbh, mby]):
108         mem += dparam * dparam
109         param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update
110
111 p += seq_length # move data pointer
112 n += 1 # iteration counter

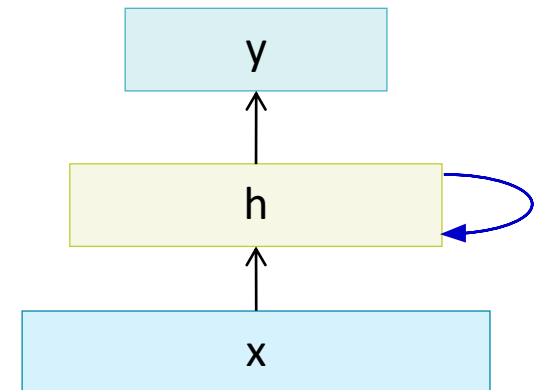
```

Shakespeare

- شامل حدود ۱۰۰,۰۰۰ کلمه

input.txt ✕

```
1 That, poor contempt, or claim'd thou slept so faithful,  
2 I may contrive our father; and, in their defeated queen,  
3 Her flesh broke me and puttance of expedition house,  
4 And in that same that ever I lament this stomach,  
5 And he, nor Butly and my fury, knowing everything  
6 Grew daily ever, his great strength and thought  
7 The bright buds of mine own.  
8  
9 BIONDELLO:  
10 Marry, that it may not pray their patience.'  
11  
12 KING LEAR:  
13 The instant common maid, as we may less be  
14 a brave gentleman and joiner: he that finds us with wax  
15 And owe so full of presence and our fooder at our  
16 staves. It is remorse'd the bridal's man his grace  
17 for every business in my tongue, but I was thinking  
18 that he contends, he hath respected thee.  
19  
20 BIRON:  
21 She left thee on, I'll die to blessed and most reasonable  
22 Nature in this honour, and her bosom is safe, some  
23 others from his speedy-birth, a bill and as  
24 Forestem with Richard in your heart  
25 Be question'd on, nor that I was enough:  
26 Which of a partier forth the obsers d'punish'd the hate
```



تکامل نمونه‌ها در حین آموزش

• تکرار ۱۰۰

tyntd-iafhatawiaoirdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e plia tkllrgd t o idoe ns,smtt h ne etie h,hregtrs
nigtike,aoaenns lng

• تکرار ۳۰۰

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh l lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

• تکرار ۷۰۰

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and offer.

• تکرار ۲۰۰۰

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.